



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 10

на тему:

«Эволюционные игры. Модель «Ястребы и голуби» и анализ
эволюционной устойчивости стратегий»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н. С. Коннова
(И. О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить эволюционные игры на примере модели «Ястребы и голуби», определить равновесия Нэша, смешанное симметричное равновесие, проверить эволюционную устойчивость стратегий и исследовать репликаторную динамику заданного варианта.

Постановка задачи:

Рассматривается симметричная эволюционная игра с двумя стратегиями:

NR – нарушать правила, стратегия «ястреб»;

R – соблюдать правила, стратегия «голубь».

Матрица выигрышей строкового игрока имеет вид:

	NR	R
NR	α	γ
R	0	β

Параметры должны удовлетворять условию:

$$\alpha < 0 < \beta < \gamma.$$

Необходимо:

1. найти чистые равновесия Нэша;
2. найти смешанное симметричное равновесие;
3. проверить эволюционную устойчивость чистых и смешанных стратегий;
4. исследовать репликаторную динамику;
5. выполнить расчёты для контрольного лекционного примера и индивидуального варианта.

Исходные данные:

Заданы параметры:

$$\alpha = -13, \beta = 5, \gamma = 18.$$

Условие модели выполняется:

$$-13 < 0 < 5 < 18.$$

Матрица выплат:

	<i>NR</i>	<i>R</i>
<i>NR</i>	(−13, −13)	(18, 0)
<i>R</i>	(0, 18)	(5, 5)

Ход работы:

Сначала были найдены чистые равновесия Нэша.

Получено два строгих равновесия:

$$(A \rightarrow NR, B \rightarrow R),$$

$$(A \rightarrow R, B \rightarrow NR).$$

Их смысл состоит в том, что если один игрок выбирает стратегию соблюдения правил *R*, то второму выгодно нарушать правила *NR*. Если же один игрок нарушает правила, то второму выгодно соблюдать правила.

Выплаты в чистых равновесиях равны:

$$(NR, R): (18, 0),$$

$$(R, NR): (0, 18).$$

Далее было найдено смешанное симметричное равновесие. Пусть x – доля стратегии *NR* в популяции. Тогда ожидаемые выигрыши чистых стратегий имеют вид:

$$U_{NR}(x) = \alpha x + \gamma(1 - x),$$

$$U_R(x) = \beta(1 - x).$$

Смешанное равновесие находится из условия:

$$U_{NR}(x^*) = U_R(x^*).$$

Отсюда:

$$x^* = \frac{\gamma - \beta}{\gamma - \beta - \alpha}.$$

Для заданных параметров:

$$x^* = \frac{18 - 5}{18 - 5 - (-13)} = \frac{13}{26} = \frac{1}{2}.$$

Следовательно, в смешанном равновесии:

$$P(NR) = \frac{1}{2},$$

$$P(R) = \frac{1}{2}.$$

Проверка выигрышей:

$$U_{NR}(x^*) = 2.5,$$

$$U_R(x^*) = 2.5.$$

Средний выигрыш смешанной стратегии против самой себя равен 2.5.

Затем была выполнена проверка эволюционной устойчивости. Чистые стратегии NR и R не являются эволюционно устойчивыми.

Стратегия NR не является ESS, так как против мутанта R :

$$u(NR, NR) = -13,$$

$$u(R, NR) = 0.$$

Поскольку $0 > -13$, мутант R может успешно вторгнуться в популяцию NR .

Стратегия R также не является ESS, так как против мутанта NR :

$$u(R, R) = 5,$$

$$u(NR, R) = 18.$$

Поскольку $18 > 5$, мутант NR может успешно вторгнуться в популяцию R .

Смешанная стратегия

$$x^* = \frac{1}{2}$$

оказалась устойчивой. Численная проверка по сетке мутантов при малой доле мутантов подтвердила, что резидентная смешанная стратегия не уступает мутантным стратегиям.

После этого была рассмотрена репликаторная динамика:

$$\frac{dx}{dt} = x(1-x)(U_{NR}(x) - U_R(x)).$$

Для заданных параметров:

$$U_{NR}(x) - U_R(x) = 13 - 26x.$$

Стационарные точки:

$$x = 0, x = \frac{1}{2}, x = 1.$$

Если $x < 0.5$, то $U_{NR}(x) > U_R(x)$, поэтому доля стратегии NR растёт.

Если $x > 0.5$, то $U_{NR}(x) < U_R(x)$, поэтому доля стратегии NR уменьшается.

Следовательно, внутренняя точка $x^* = 0.5$ является устойчивой, а точки $x = 0, x = 1$ являются неустойчивыми.

Графики подтверждают этот результат: траектории репликаторной динамики направлены к точке $x^* = 0.5$.

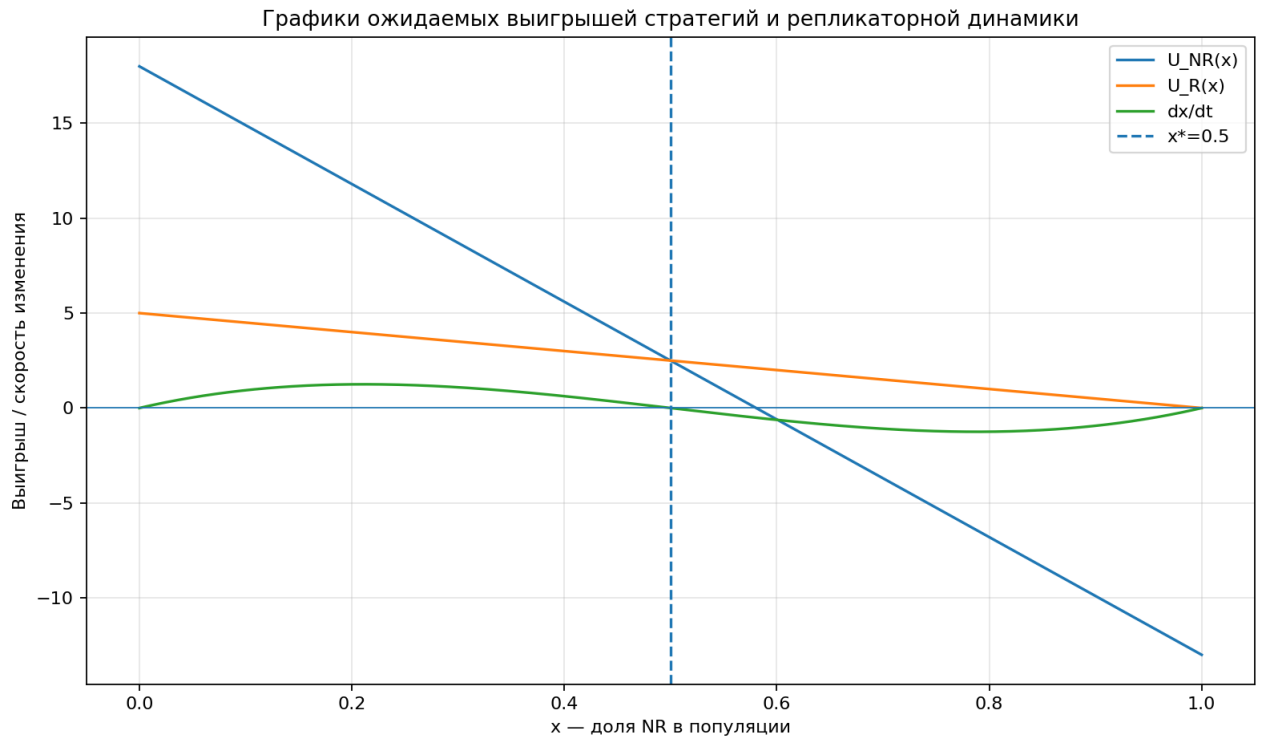


Рисунок 1 – Графики ожидаемых выигрышей стратегий и репликаторной динамики

Вывод:

В ходе работы была рассмотрена эволюционная игра «Ястребы и голуби». Были найдены чистые равновесия Нэша, смешанное симметричное равновесие и исследована эволюционная устойчивость стратегий.

Для варианта с параметрами

$$\alpha = -13, \beta = 5, \gamma = 18$$

найжены два чистых равновесия Нэша:

$$(NR, R),$$

$$(R, NR).$$

Чистые стратегии NR и R не являются эволюционно устойчивыми.

Смешанная стратегия

$$x^* = \frac{1}{2}$$

является устойчивой. Это означает, что в долгосрочном поведении популяции устанавливается равновесное соотношение:

50% стратегии NR ,

50% стратегии R .

Репликаторная динамика показала, что при начальных значениях $x < 0.5$ доля стратегии NR возрастает, а при $x > 0.5$ уменьшается. Следовательно, внутренняя точка равновесия

$$x^* = 0.5$$

является устойчивым эволюционным состоянием.

Приложение (исходный код)

```
from __future__ import annotations
from dataclasses import dataclass
from fractions import Fraction
from pathlib import Path
from typing import Iterable, List, Sequence, Tuple

STRATEGIES = ("NR", "R")
STRATEGY_NAMES = {
    "NR": "нарушать правила / ястреб",
    "R": "соблюдать правила / голубь",
}

def fmt_float(x: float, digits: int = 6) -> str:
    if abs(x - round(x)) < 10 ** (-(digits - 1)):
        return str(int(round(x)))
    return f"{x:.{digits}f}".rstrip("0").rstrip(".")

def fmt_fraction(fr: Fraction) -> str:
    if fr.denominator == 1:
        return str(fr.numerator)
    return f"{fr.numerator}/{fr.denominator} ≈ {fmt_float(float(fr))}"

def line(char: str = "-", width: int = 96) -> None:
    print(char * width)

@dataclass(frozen=True)
class SymmetricTwoStrategyGame:
    strategies: Tuple[str, str]
    payoff_matrix: Tuple[Tuple[float, float], Tuple[float, float]]
    alpha: float
    beta: float
    gamma: float
    title: str

    def payoff(self, row_strategy: str, col_strategy: str) -> float:
        i = self.strategies.index(row_strategy)
        j = self.strategies.index(col_strategy)
        return self.payoff_matrix[i][j]

    def pair_payoff(self, row_strategy: str, col_strategy: str) -> Tuple[float, float]:
        u1 = self.payoff(row_strategy, col_strategy)
        u2 = self.payoff(col_strategy, row_strategy)
        return u1, u2

    def payoff_against_share_nr(self, own_strategy: str, share_nr: float) -> float:
        return (
            share_nr * self.payoff(own_strategy, "NR")
            + (1.0 - share_nr) * self.payoff(own_strategy, "R")
        )

    def expected_payoff_mixed(self, own_share_nr: float, opp_share_nr: float) -> float:
        p = own_share_nr
        q = opp_share_nr
        a00 = self.payoff("NR", "NR")
        a01 = self.payoff("NR", "R")
        a10 = self.payoff("R", "NR")
        a11 = self.payoff("R", "R")
        return p * q * a00 + p * (1 - q) * a01 + (1 - p) * q * a10 + (1 - p) * (1 - q) * a11

    def delta_nr_minus_r(self, share_nr: float) -> float:
        return self.payoff_against_share_nr("NR", share_nr) - self.payoff_against_share_nr("R",
share_nr)

    def threshold_share_nr_fraction(self) -> Fraction:
        a = Fraction(str(self.alpha))
        b = Fraction(str(self.beta))
        g = Fraction(str(self.gamma))
        return (g - b) / (g - b - a)

    def threshold_share_nr(self) -> float:
        return float(self.threshold_share_nr_fraction())

    def validate_hawk_dove_assumptions(self) -> None:
        if not (self.alpha < 0 < self.beta < self.gamma):
            raise ValueError(
                "Для модели 'Ястребы и голуби' должно выполняться alpha < 0 < beta < gamma. "
```

```

        f"Сейчас: alpha={self.alpha}, beta={self.beta}, gamma={self.gamma}"
    )

def build_hawk_dove_game(alpha: float, beta: float, gamma: float, title: str) ->
SymmetricTwoStrategyGame:
    game = SymmetricTwoStrategyGame(
        strategies=STRATEGIES,
        payoff_matrix=((alpha, gamma), (0.0, beta)),
        alpha=alpha,
        beta=beta,
        gamma=gamma,
        title=title,
    )
    game.validate_hawk_dove_assumptions()
    return game

def build_individual_variant(variant: int = 13) -> SymmetricTwoStrategyGame:
    alpha = -variant
    beta = 2 + (variant % 5)
    gamma = beta + variant
    return build_hawk_dove_game(
        alpha=alpha,
        beta=beta,
        gamma=gamma,
        title=f"Индивидуальный вариант {variant}",
    )

def best_responses_to_column(game: SymmetricTwoStrategyGame, col_strategy: str) -> List[str]:
    values = {s: game.payoff(s, col_strategy) for s in game.strategies}
    best = max(values.values())
    return [s for s, value in values.items() if abs(value - best) <= 1e-9]

def best_responses_to_row_for_column(game: SymmetricTwoStrategyGame, row_strategy: str) ->
List[str]:
    values = {s: game.payoff(s, row_strategy) for s in game.strategies}
    best = max(values.values())
    return [s for s, value in values.items() if abs(value - best) <= 1e-9]

def pure_nash_equilibria(game: SymmetricTwoStrategyGame) -> List[Tuple[str, str, bool]]:
    equilibria: List[Tuple[str, str, bool]] = []
    for row in game.strategies:
        for col in game.strategies:
            br_row = best_responses_to_column(game, col)
            br_col = best_responses_to_row_for_column(game, row)
            if row in br_row and col in br_col:
                strict = len(br_row) == 1 and len(br_col) == 1
                equilibria.append((row, col, strict))
    return equilibria

def mixed_symmetric_equilibrium(game: SymmetricTwoStrategyGame) -> Fraction:
    return game.threshold_share_nr_fraction()

def pure_strategy_ess_status(game: SymmetricTwoStrategyGame, resident: str) -> Tuple[bool,
List[str]]:
    comments: List[str] = []
    ok = True
    for mutant in game.strategies:
        if mutant == resident:
            continue
        u_ss = game.payoff(resident, resident)
        u_ms = game.payoff(mutant, resident)
        u_sm = game.payoff(resident, mutant)
        u_mm = game.payoff(mutant, mutant)
        if u_ss > u_ms + 1e-9:
            comments.append(
                f"против мутанта {mutant}: u({resident},{resident})={fmt_float(u_ss)} > "
                f"u({mutant},{resident})={fmt_float(u_ms)}"
            )
        elif abs(u_ss - u_ms) <= 1e-9 and u_sm > u_mm + 1e-9:
            comments.append(
                f"против мутанта {mutant}: первая проверка дала равенство, но "
                f"u({resident},{mutant})={fmt_float(u_sm)} > "
                f"u({mutant},{mutant})={fmt_float(u_mm)}"
            )
        else:
            comments.append(
                f"против мутанта {mutant}: условие ESS нарушено, так как "
                f"u({resident},{resident})={fmt_float(u_ss)}, "
                f"u({mutant},{resident})={fmt_float(u_ms)}"
            )
    return (ok, comments)

```

```

        )
        ok = False
    return ok, comments

def mixed_ess_grid_check(
    game: SymmetricTwoStrategyGame,
    resident_share_nr: float,
    eps: float = 0.01,
    grid_size: int = 1001,
) -> Tuple[bool, float, float]:
    min_diff = float("inf")
    arg_min = 0.0
    ok = True
    for k in range(grid_size):
        mutant_share_nr = k / (grid_size - 1)
        if abs(mutant_share_nr - resident_share_nr) <= 1e-6:
            continue
        population_share_nr = (1 - eps) * resident_share_nr + eps * mutant_share_nr
        u_resident = game.expected_payoff_mixed(resident_share_nr, population_share_nr)
        u_mutant = game.expected_payoff_mixed(mutant_share_nr, population_share_nr)
        diff = u_resident - u_mutant
        if diff < min_diff:
            min_diff = diff
            arg_min = mutant_share_nr
        if diff <= -1e-8:
            ok = False
    return ok, min_diff, arg_min

def replicator_rhs(game: SymmetricTwoStrategyGame, share_nr: float) -> float:
    x = share_nr
    return x * (1 - x) * game.delta_nr_minus_r(x)

def simulate_replicator(
    game: SymmetricTwoStrategyGame,
    x0: float,
    steps: int = 25,
    dt: float = 0.05,
) -> List[Tuple[int, float, float, float, float]]:
    x = x0
    rows: List[Tuple[int, float, float, float, float]] = []
    for step in range(steps + 1):
        u_nr = game.payoff_against_share_nr("NR", x)
        u_r = game.payoff_against_share_nr("R", x)
        dx = replicator_rhs(game, x)
        rows.append((step, x, u_nr, u_r, dx))
        x = x + dt * dx
        x = min(1.0, max(0.0, x))
    return rows

def print_payoff_matrix(game: SymmetricTwoStrategyGame) -> None:
    print("Матрица выплат в симметричной игре. В клетке указано (выигрыш A, выигрыш B):")
    print()
    print("          B: NR          B: R")
    for row in game.strategies:
        cells = []
        for col in game.strategies:
            u1, u2 = game.pair_payoff(row, col)
            cells.append(f"({fmt_float(u1)}, {fmt_float(u2)})".rjust(16))
        print(f"A: {row:<2} {cells[0]} {cells[1]}")
    print()
    print("Матрица выигрышей строкового игрока A:")
    print("          NR          R")
    for row in game.strategies:
        print(
            f"{row:<4}"
            f"{fmt_float(game.payoff(row, 'NR')).rjust(10)}"
            f"{fmt_float(game.payoff(row, 'R')).rjust(11)}"
        )

def print_pure_nash(game: SymmetricTwoStrategyGame) -> None:
    line()
    print("1. Чистые равновесия Нэша")
    eqs = pure_nash_equilibria(game)
    if not eqs:
        print("Чистых равновесий Нэша нет.")
        return
    print(f"Найдено чистых равновесий Нэша: {len(eqs)}")
    for idx, (row, col, strict) in enumerate(eqs, start=1):
        u1, u2 = game.pair_payoff(row, col)

```

```

        strict_text = "строгое" if strict else "нестрогое"
    print(
        f" {idx}) (A->{row}, B->{col}), выплаты = ({fmt_float(u1)}, {fmt_float(u2)}), "
        f"{strict_text} равновесие"
    )

print("\nПроверка лучших ответов:")
for row in game.strategies:
    for col in game.strategies:
        br_row = best_responses_to_column(game, col)
        br_col = best_responses_to_row_for_column(game, row)
        mark = "NE" if any(row == r and col == c for r, c, _ in eqs) else " "
        print(
            f" {mark} A->{row}, B->{col}: "
            f"BR_A({col})={br_row}, BR_B({row})={br_col}"
        )

def print_mixed_equilibrium(game: SymmetricTwoStrategyGame) -> None:
    line()
    print("2. Смешанное симметричное равновесие")
    p = mixed_symmetric_equilibrium(game)
    p_float = float(p)
    u_nr = game.payoff_against_share_nr("NR", p_float)
    u_r = game.payoff_against_share_nr("R", p_float)
    u_mix = game.expected_payoff_mixed(p_float, p_float)
    print("Пусть x - доля стратегии NR в популяции или вероятность выбора NR.")
    print("Ожидаемые выигрыши чистых стратегий против популяции с долей x:")
    print(
        f" U_NR(x) = alpha*x + gamma*(1-x) "
        f" = {fmt_float(game.alpha)}*x + {fmt_float(game.gamma)}*(1-x) "
    )
    print(
        f" U_R(x) = 0*x + beta*(1-x) "
        f" = {fmt_float(game.beta)}*(1-x) "
    )
    print("Смешанное равновесие находится из условия U_NR(x*) = U_R(x*):")
    print(" x* = (gamma - beta) / (gamma - beta - alpha)")
    print(f" x* = {fmt_fraction(p)}")
    print(f" Вероятность NR: {fmt_fraction(p)}")
    print(f" Вероятность R : {fmt_fraction(1 - p)}")
    print("Проверка равенства выигрышей в смешанном равновесии:")
    print(f" U_NR(x*) = {fmt_float(u_nr)}")
    print(f" U_R(x*) = {fmt_float(u_r)}")
    print(f" Средний выигрыш смешанной стратегии против самой себя = {fmt_float(u_mix)}")

def print_mutant_analysis(game: SymmetricTwoStrategyGame) -> None:
    line()
    print("3. Анализ мутантов и эволюционная устойчивость")

    p = game.threshold_share_nr_fraction()
    p_float = float(p)

    print("В лекции используется доля мутантов ε. Если в популяции доля NR равна ε,")
    print("то нарушать правила выгодно при условии:")
    print(f" ε*alpha + (1-ε)*gamma >= (1-ε)*beta")
    print("После преобразования:")
    print(f" ε <= (gamma - beta) / (gamma - beta - alpha)")
    print(f" ε <= {fmt_fraction(p)}")

    print("\nПроверка чистых стратегий на ESS:")
    for resident in game.strategies:
        ok, comments = pure_strategy_ess_status(game, resident)
        print(f" Стратегия {resident} ({STRATEGY_NAMES[resident]}): {'ESS' if ok else 'не ESS'}")
        for comment in comments:
            print(f" - {comment}")

    print("\nПроверка смешанной стратегии x* на устойчивость против мутантов.")
    ok_grid, min_diff, arg_min = mixed_ess_grid_check(game, p_float, eps=0.01, grid_size=2001)
    print(f" x* = {fmt_fraction(p)}")
    print(
        f" Численная проверка по сетке мутантов при ε=0.01: "
        f"{'устойчива' if ok_grid else 'есть нарушение'}"
    )
    print(
        f" Минимальная разность U_resident - U_mutant на сетке: "
        f"{fmt_float(min_diff)} при p_mutant={fmt_float(arg_min)}"
    )

```

```

print("\nТаблица устойчивости смешанной стратегии против нескольких мутантов:")
print("  p_mutant - вероятность NR у мутантной стратегии")
print("  eps      p_mutant      U_resident      U_mutant      difference")
for eps in (0.01, 0.05, 0.10):
    for p_mutant in (0.0, 0.25, 0.75, 1.0):
        population_share = (1 - eps) * p_float + eps * p_mutant
        u_resident = game.expected_payoff_mixed(p_float, population_share)
        u_mutant = game.expected_payoff_mixed(p_mutant, population_share)
        diff = u_resident - u_mutant
        print(
            f"    {fmt_float(eps).rjust(5)}"
            f"    {fmt_float(p_mutant).rjust(15)}"
            f"    {fmt_float(u_resident).rjust(17)}"
            f"    {fmt_float(u_mutant).rjust(15)}"
            f"    {fmt_float(diff).rjust(15)}"
        )

def print_replicator_analysis(game: SymmetricTwoStrategyGame) -> None:
    line()
    print("4. Репликационная динамика")

    p = game.threshold_share_nr_fraction()
    p_float = float(p)

    print("Пусть x(t) - доля стратегии NR в популяции.")
    print("Репликационная динамика для двух стратегий:")
    print("  dx/dt = x(1-x) (U_NR(x) - U_R(x))")
    print("Для данной игры:")
    print(
        "  U_NR(x) - U_R(x) = "
        f"  (gamma - beta) + x*(alpha - gamma + beta) = "
        f"  {fmt_float(game.gamma - game.beta)} + x*({fmt_float(game.alpha - game.gamma + game.beta)})"
    )
    print("Стационарные точки:")
    print("  x = 0")
    print(f"  x = x* = {fmt_float(p)}")
    print("  x = 1")

    print("Устойчивость:")
    print("  при x < x*: U_NR(x) > U_R(x), поэтому доля NR растет;")
    print("  при x > x*: U_NR(x) < U_R(x), поэтому доля NR уменьшается;")
    print("следовательно, внутренняя точка x* устойчива, а x=0 и x=1 неустойчивы.")

    print("\nКороткая имитация методом Эйлера:")
    for x0 in (0.10, max(0.01, p_float - 0.20), min(0.99, p_float + 0.20), 0.90):
        rows = simulate_replicator(game, x0=x0, steps=10, dt=0.05)
        print(f"\n Начальное значение x0={fmt_float(x0)}")
        print("  step      x      U_NR      U_R      dx/dt")
        for step, x, u_nr, u_r, dx in rows:
            print(
                f"    {str(step).rjust(4)}"
                f"    {fmt_float(x).rjust(11)}"
                f"    {fmt_float(u_nr).rjust(12)}"
                f"    {fmt_float(u_r).rjust(12)}"
                f"    {fmt_float(dx).rjust(13)}"
            )

def save_optional_plot(game: SymmetricTwoStrategyGame, output_path: str) -> None:
    try:
        import matplotlib.pyplot as plt
    except Exception:
        print("\nmatplotlib не установлен, график не построен.")
        return

    p = game.threshold_share_nr()
    xs = [i / 500 for i in range(501)]
    u_nr = [game.payoff_against_share_nr("NR", x) for x in xs]
    u_r = [game.payoff_against_share_nr("R", x) for x in xs]
    dx = [replicator_rhs(game, x) for x in xs]

    output = Path(output_path)
    if not output.is_absolute():
        output = Path(__file__).resolve().parent / output
    output.parent.mkdir(parents=True, exist_ok=True)

    plt.figure(figsize=(10, 6))
    plt.plot(xs, u_nr, label="U_NR(x)")
    plt.plot(xs, u_r, label="U_R(x)")

```

```

plt.plot(xs, dx, label="dx/dt")
plt.axvline(p, linestyle="--", label=f"x*={fmt_float(p)}")
plt.axhline(0, linewidth=0.8)
plt.title(game.title)
plt.xlabel("x - доля NR в популяции")
plt.ylabel("Выигрыш / скорость изменения")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig(output, dpi=160)
plt.close()
print(f"\nГрафик сохранен: {output}")

def analyze_game(game: SymmetricTwoStrategyGame, make_plot: bool = False, plot_path: str | None =
None) -> None:
    line("")
    print(game.title)
    line("")
    print(f"Параметры: alpha={fmt_float(game.alpha)}, beta={fmt_float(game.beta)},
gamma={fmt_float(game.gamma)}")
    print("Проверка условия модели:  $\alpha < 0 < \beta < \gamma$  - выполнено")
    print()
    print_payoff_matrix(game)
    print_pure_nash(game)
    print_mixed_equilibrium(game)
    print_mutant_analysis(game)
    print_replicator_analysis(game)

    if make_plot:
        if plot_path is None:
            safe_title = game.title.replace(" ", " ").replace("/", " ")
            plot_path = f"data/{safe_title}.png"
            save_optional_plot(game, plot_path)

def print_lecture_symbolic_reference() -> None:
    line("#")
    print("ЛЕКЦИОННЫЙ ПРИМЕР: структура игры 'Ястребы и голуби'")
    line("#")
    print("В лекции заданы стратегии:")
    print("NR - нарушать правила ('ястреб')")
    print("R - соблюдать правила ('голубь')")
    print("и параметры  $\alpha < 0 < \beta < \gamma$ .")
    print("Матрица выигрышей строкового игрока:")
    print("      NR      R")
    print("NR   alpha  gamma")
    print("R     0      beta")
    print("Из этой матрицы следует:")
    print("чистые равновесия Нэша: (NR, R) и (R, NR);")
    print("нарушать выгодно при  $\epsilon \leq (\gamma - \beta) / (\gamma - \beta - \alpha)$ ");
    print("смешанная эволюционно устойчивая доля NR равна этому же порогу.")
    print()

def main() -> None:
    VARIANT = 13

    print_lecture_symbolic_reference()

    lecture_check_game = build_hawk_dove_game(
        alpha=-4,
        beta=2,
        gamma=6,
        title="Контрольный числовой пример по структуре лекции: alpha=-4, beta=2, gamma=6",
    )
    analyze_game(
        lecture_check_game,
        make_plot=True,
        plot_path="data/evolution_lecture_check.png",
    )

    individual_game = build_individual_variant(VARIANT)
    analyze_game(
        individual_game,
        make_plot=True,
        plot_path=f"data/evolution_variant_{VARIANT}.png",
    )

if __name__ == "__main__":
    main()

```